

The Agentic AI Operating Model & Agent FinOps

Day 81 was the regulator's clock; Day 82 was the buyer's checklist. Today is what happens **after you sign** — you have to **run, cost and scale** the fleet you just bought. Last week's **FinOps X 2026** made it official: **98% of FinOps teams now manage AI spend, up from 31% two years ago**. The discipline that used to govern cloud bills is being rebuilt around **tokens, agents and outcomes** — and most organisations still can't see where the money goes. This issue is the operating model: why agent cost is non-deterministic, the shift from *cost per token* to *cost per outcome*, the five levers that keep a fleet from running away, and how the same **audit trail you procured** (Day 82) becomes your live cost-and-control dashboard.

98% of FinOps practitioners now manage AI spend, up from 31% two years ago (State of FinOps 2026)	20K → 2M tokens a single agent prompt can burn depending on design = AI cost is non-deterministic (AWS)	4× / \$30M throughput lift + savings from Google's invoice-reconciliation agent (humans review, not do)	60–80% of fleet work agents handle by volume; humans keep the high-stakes 20–40%
---	---	---	--

1 · Why the cloud playbook breaks — the cost iceberg

Tokenomics is the visible tip. Below the waterline, when an agent executes a task it may **spin up a VM in a sandbox** to run scripts, **consume key-value cache storage**, and **trigger RAG pipelines** — costs that sit entirely outside the input/output token line item. Google Cloud's Pravir Gupta called it exactly that at FinOps X: *"It's like the iceberg — what's under the water."* Worse, the spend is **non-deterministic**: AWS's Jerry Rapisarda noted a single prompt to an agent could consume **20,000 tokens or two million**, depending on how the system is designed. That is why tagging an EC2 instance no longer covers it. The scaling math is brutal — an agent running at ~\$5,000/month with 1,000 daily conversations can realistically hit **\$20,000–\$25,000/month at 10,000 users** once reasoning tokens, state management and tool calls stack up. At production scale, **agent operating spend becomes the largest line item in the monthly budget** — and the one nobody owned a year ago.

***The reframe:** you are not running a chatbot, you are running a fleet of non-deterministic workers that each spin up infrastructure on demand. FinOps for AI is now the iceberg-management discipline — and 98% of teams are in it whether they planned to be or not.*

2 · The unit that matters: cost per outcome, not cost per token

The C-suite question for 2026 is blunt: **how much does an agent cost per successfully completed task?** Cost per token is an input metric; what matters is cost per *invocation* tied to a *business outcome*. Rapisarda's example: a chatbot at **3¢ per invocation** converting at **4%** is only meaningful once you know what that conversion drives in revenue. SailPoint's Victoria Levy put it plainly at FinOps X: *"The KPIs are going to be way different — cost per token, tokens per business driver ... it will converge on things that are useful."* This is the same unit this series built on Day 63 (revenue-per-task × task-completion-rate) and Day 49 (the Cost-per-Task SLO). The table below is the translation every FinOps team is now writing.

Old cloud KPI	New agent KPI	Why it changed
Cost per instance-hour	Cost per completed task (total spend ÷ tasks that hit the goal, not tasks attempted)	A task can loop, retry or fail silently; only completed-and-correct work creates value.

Old cloud KPI	New agent KPI	Why it changed
Utilisation %	Cost per outcome × quality (3¢/invocation at 4% conversion = \$X revenue)	Spend is only good or bad relative to the business outcome it produces.
Reserved-capacity coverage	Token mix by model tier (% nano / mid / frontier) + cache-hit rate	Routing and caching, not reservations, are where agent savings live (Day 43).
Monthly bill by service	Cost granularity by orchestrator → sub-agent → model → org tag	Headless orchestrators call sub-agents on different tiers; you must attribute every hop.

3 · The operating model — five levers that keep a fleet from running away

Once you can see cost per outcome, five levers control it. None is new to this series; what's new is that FinOps X 2026 made them a named discipline with tooling behind them.

Lever	What it does	Maps to
1. Model routing	Send classification + simple tools to a nano/flash model, code + reasoning to a mid tier, only ambiguous planning to frontier. Lands at ~15% of all-frontier cost at indistinguishable quality.	Day 43 (70/25/5)
2. Caching + context discipline	Reuse KV cache; compress memory. Newer text-memory systems (e.g. Mastra Observational Memory) cut token cost 4–10× vs naive context stuffing.	Day 39 / 44
3. Budget caps + circuit breakers	Hard per-task / per-fleet spend ceilings and loop detection that trips before a stuck ReAct loop burns real money. A runaway loop is now a cost incident.	Day 24 / 49
4. Tagging + cost granularity	High-quality attribution tags per orchestrator, sub-agent, model and team. Datadog's lesson: without tags, the ability to allocate spend and find waste collapses.	Day 22 / 82
5. Human-in-the-loop review	Humans review agent output rather than do the work. Google's invoice agent hit 4× throughput + \$30M savings this way — the trick is review + feedback, not 100% autonomy.	Day 48 / 70

***Enforcement, not advice.** Levy's warning from the FinOps X floor: tell people to right-size once and the savings erode the moment you look away. The levers only hold if they're automated and enforced — budget caps in code, routing in the gateway, tags required at deploy — not a quarterly slide deck.*

4 · Who runs it — the Agent SRE + CFO + FinOps triangle

Agent FinOps is a team sport, and the clearest signal from FinOps X was that **cross-functional context decides whether a cost is waste or an intentional trade-off** (a pricey call might be a security requirement, not sloppiness). Three roles share the fleet, and the tooling is arriving to back them: **AWS's FinOps Agent** (preview) autonomously monitors spend, detects anomalies, does root-cause analysis and routes alerts to the owning team in Slack/Jira without waiting for end-of-month — itself built *human-in-the-loop* because, as AWS put it, “there's a trust arc to be earned around full autonomous actions.” Bedrock now lets you compare cost-per-token across models and allocate by IAM role, so sandbox and production get different model tiers by policy.

Role	Owns	The question they ask
Agent SRE	Fleet SLOs: task success, quality, cost-per-task , safety. Burn-rate alerts + kill switch. (Day 49)	“Is this fleet within its cost and quality error budget right now?”

Role	Owns	The question they ask
CFO / Finance	ROI and unit economics: cost per outcome vs revenue per outcome; the pre-committed token budget. (Day 31/63)	"What does a completed task cost us, and what does it earn?"
FinOps / Platform	Attribution, tagging, routing policy, enforcement; the AI gateway and cost dashboard. (Day 82)	"Can we attribute every token to a team — and is the saving enforced?"

***The convergence:** the OTEL→WORM audit trail you procured for the regulator (Day 81) and the buyer (Day 82) is the same telemetry stream that powers your cost dashboard. Governance and FinOps are one pipeline: every model call, tool call and approval gate is both an audit event and a cost event. And pricing closes the loop — per-seat is dying; frontier providers are moving to **seat-fee + pre-committed token consumption** (Anthropic's April 2026 enterprise transition is the front edge), so your routing and caching discipline is now what you negotiate against.*

5 · Viral app spotlight — agenttrace, FinOps for your own desk

On a day about cost-and-control dashboards, the trending tool is the one that puts that dashboard on a single developer's machine. **agenttrace** ([luoyuctl/agenttrace](#), v0.5.2) is a **local-first terminal TUI** that reads your AI coding-agent session logs — Claude Code, Codex CLI, Gemini CLI, Qwen Code, Cline, Aider, Cursor, Hermes Agent, OpenCode, OpenClaw, Kimi CLI and generic JSON/JSONL — and traces **cost, input/output/cache tokens, wall-clock time, tool failures, retries, hanging runs, health scores and diffs**, then emits JSON / Markdown / HTML reports with **health and tool-failure gates you can wire into CI**. It's the personal-scale mirror of everything in this issue: the cost iceberg, per-task attribution, and a kill-gate on runaway loops. It's riding a whole micro-wave of agent-cost observability tools — **codeburn** (`npx codeburn`, "see where your AI tokens go") and **agentwatch** (one local timeline across coding and non-coding agents) launched in the same window. (OpenClaw still tops the raw OSS star charts at 210K+ as the local-first foil — viral, but it ships with no cost meter, no kill switch and no scoped identity: the exact blind-spot this issue is about.)

Local-first reads 12+ agent log formats on your machine, no API key, no data egress	Cost + tokens per session by source, model, in/out/cache tokens, time, tool failures	CI gates health + tool-failure gates in JSON/MD/HTML = a cost kill-gate for your agents
---	--	---

Practical takeaways

1. Measure cost per outcome, not cost per token.

Instrument OTEL gen_ai spans, tag every call by orchestrator, sub-agent, model and team, and divide spend by *completed* tasks. A token count without a business outcome attached tells you nothing about whether you're winning.

2. Wire the five levers before you scale — and enforce them in code.

Routing (70/25/5), caching/context discipline, budget caps + circuit breakers, required tags, and human-in-the-loop review. Right-sizing once is theatre; the savings only stick when the levers are automated and enforced at the gateway, not recommended in a deck.

3. Make it somebody's job — stand up the Agent SRE + CFO + FinOps triangle.

Point all three at one OTEL→WORM stream so the audit trail you bought doubles as the cost dashboard. With frontier models within ~3% of each other, you can't out-model a rival — but you can out-operate them by running the lowest cost per successful, audited outcome.

***Tomorrow (Day 84):** The Agent-Augmented Org Chart — who actually runs the fleet. Team topologies, RACI and the new roles (Agent SRE, AI FinOps lead, Context Engineer, the human-review function) that a traditional org re-wires into once agents become coworkers, not tools.*